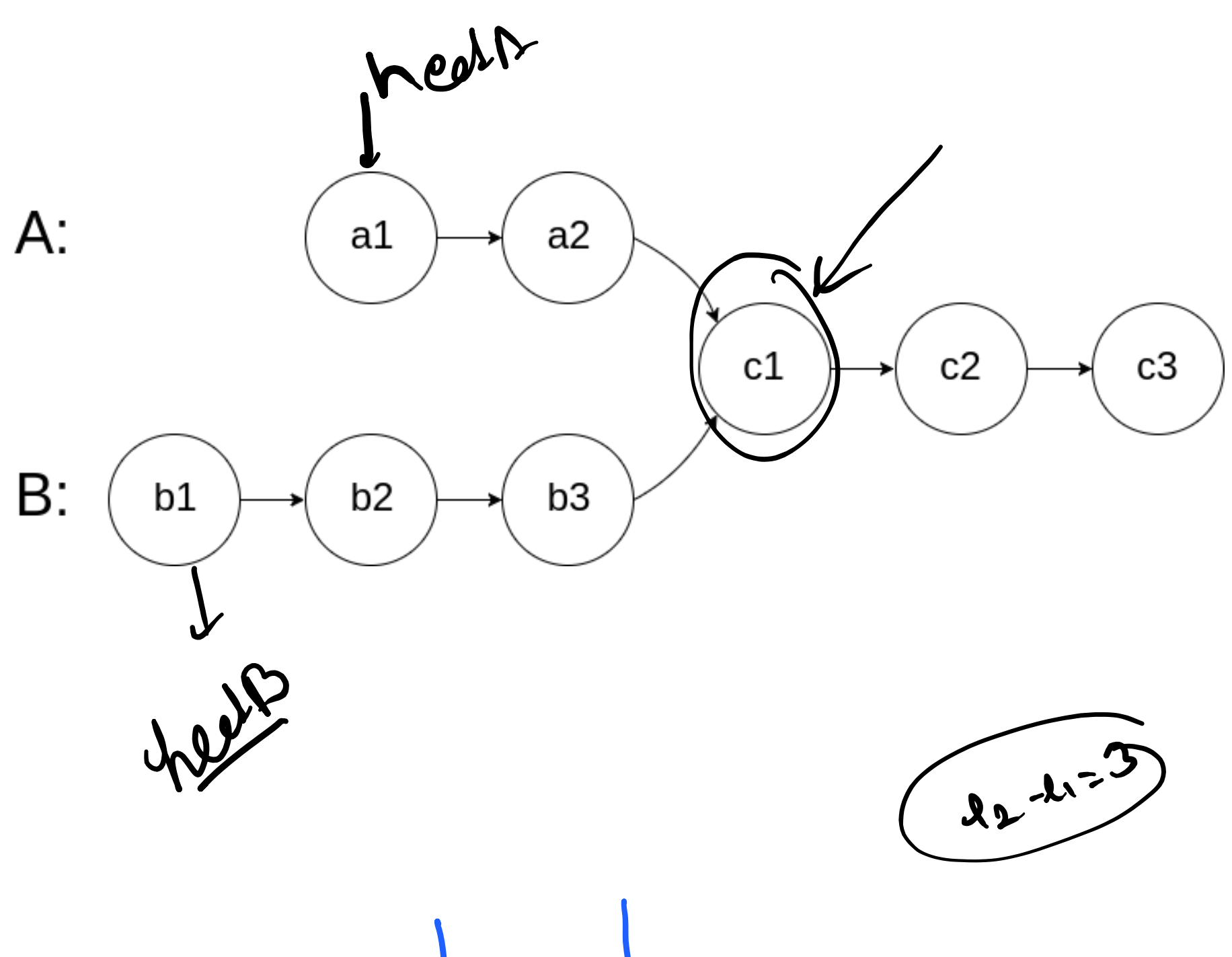
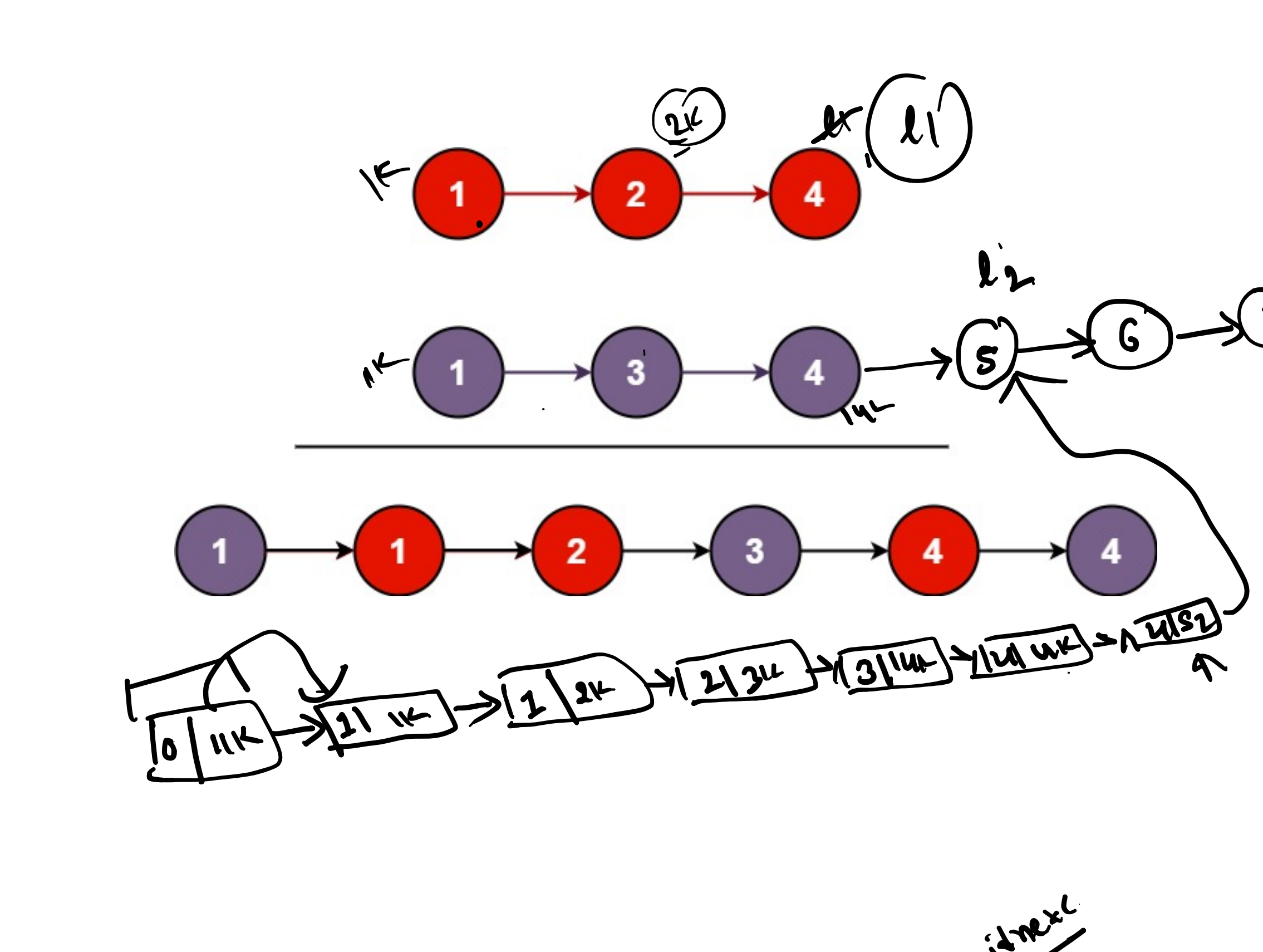
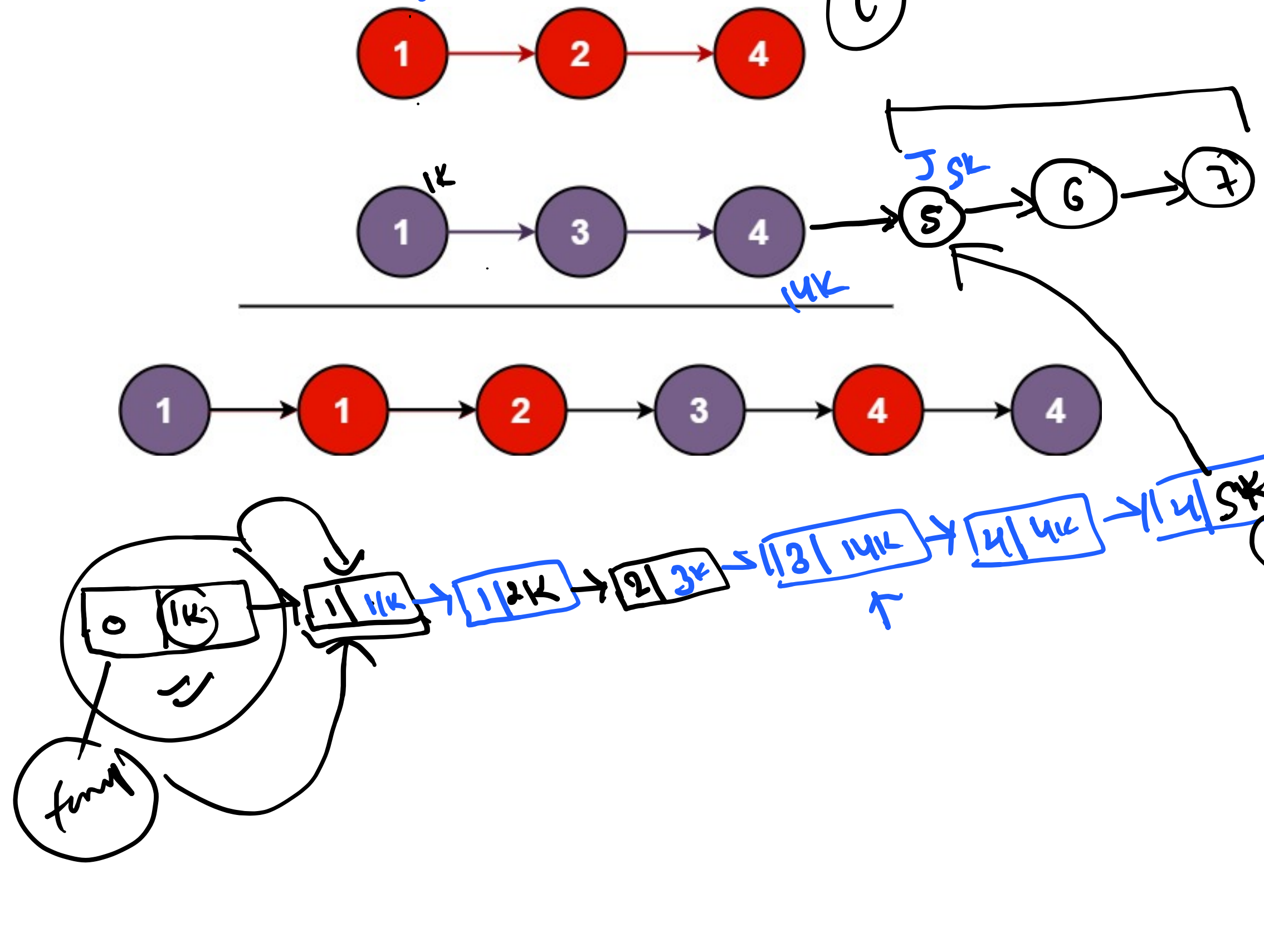
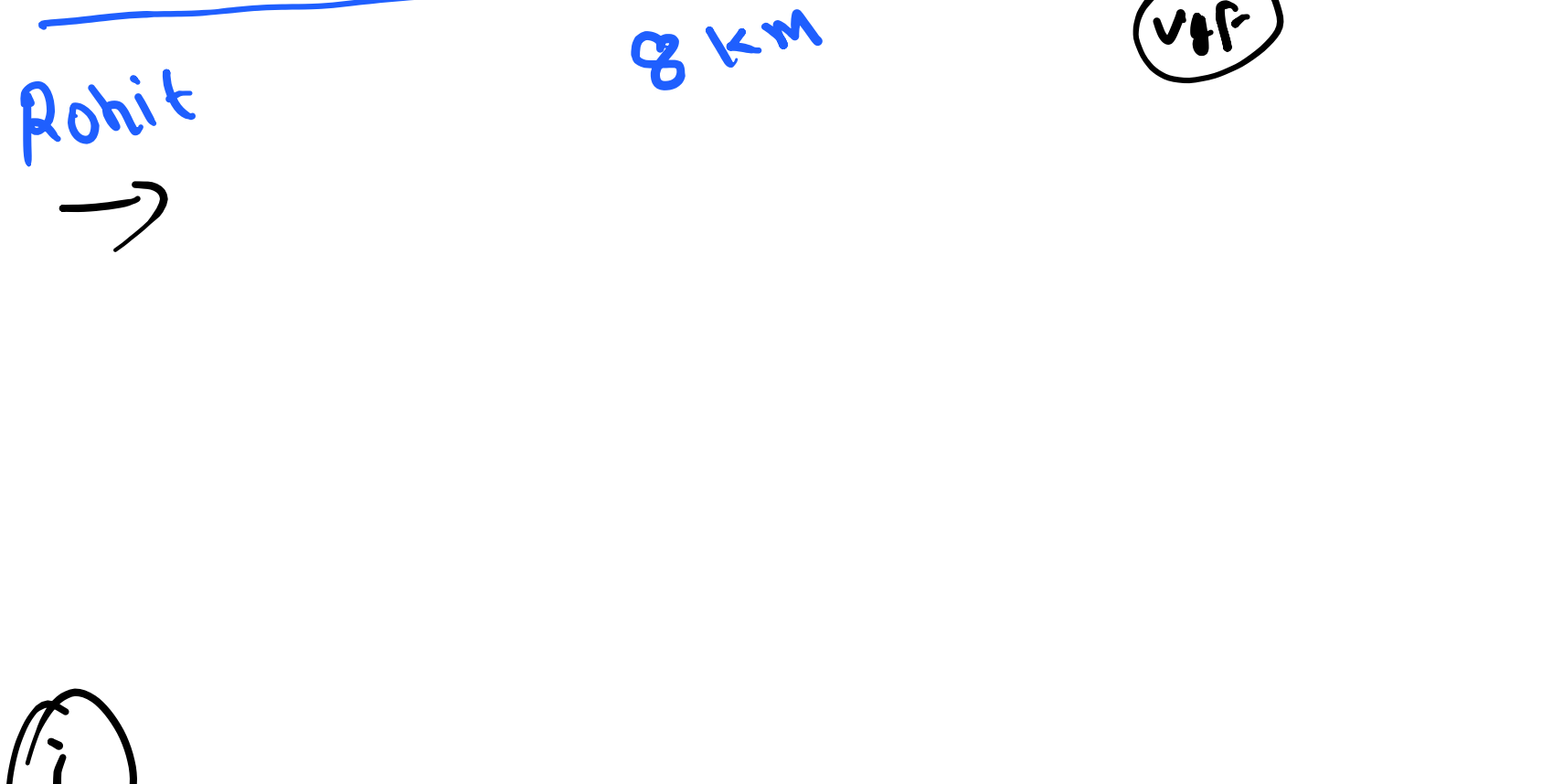
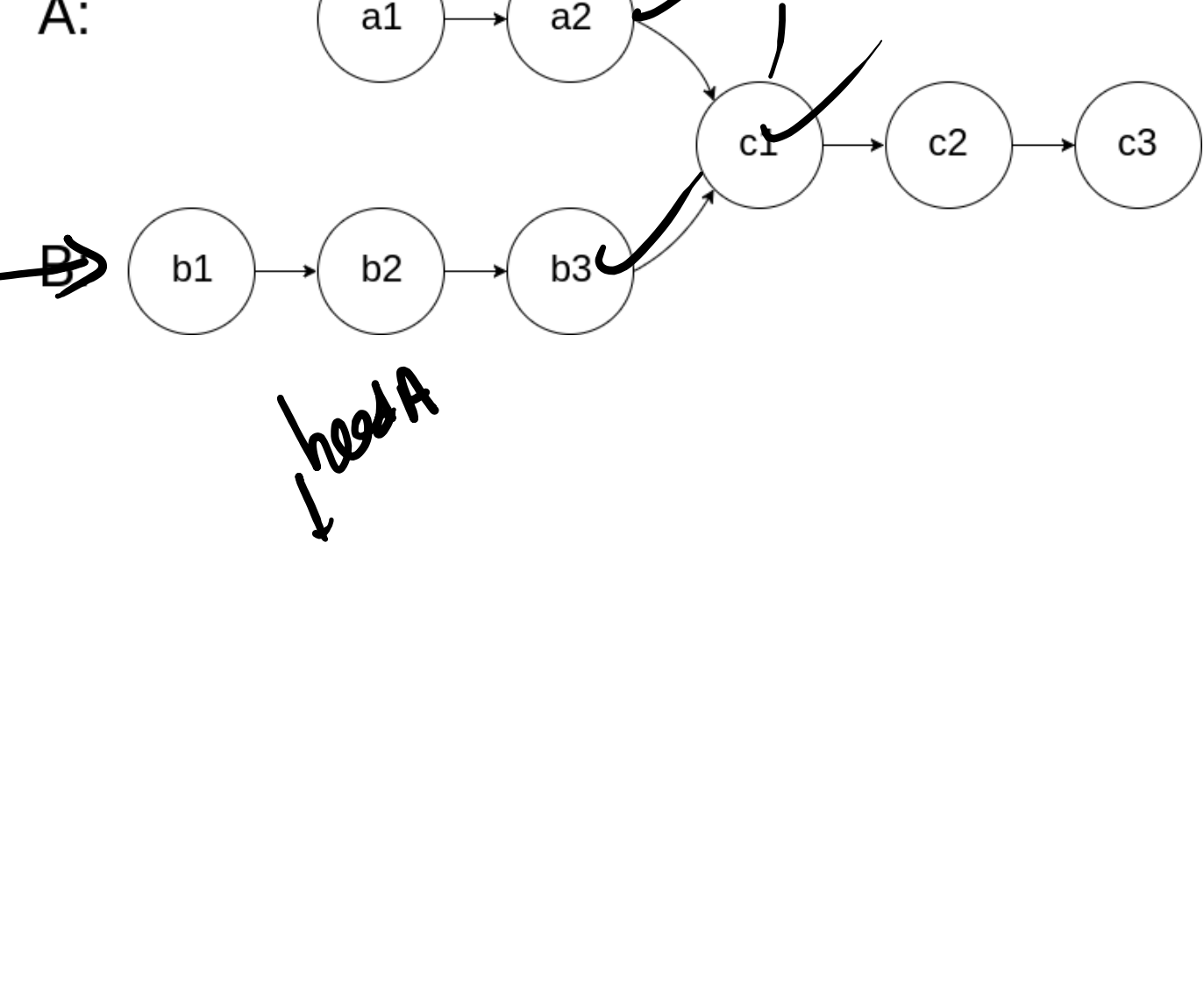




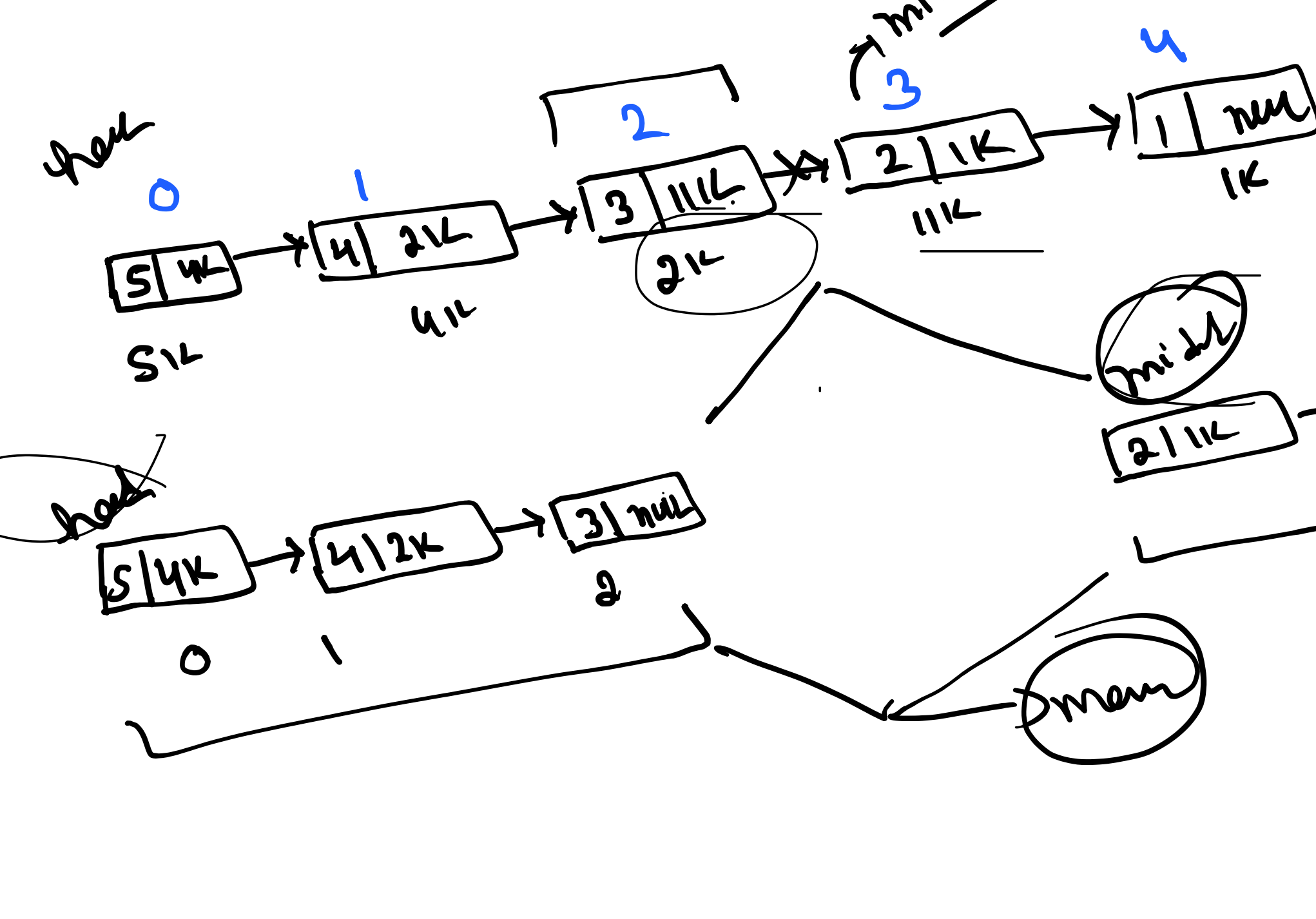
$d_2 - d_1 = 3$
 $d_1 = 5$
 $d_2 = 8$
 $length \times$



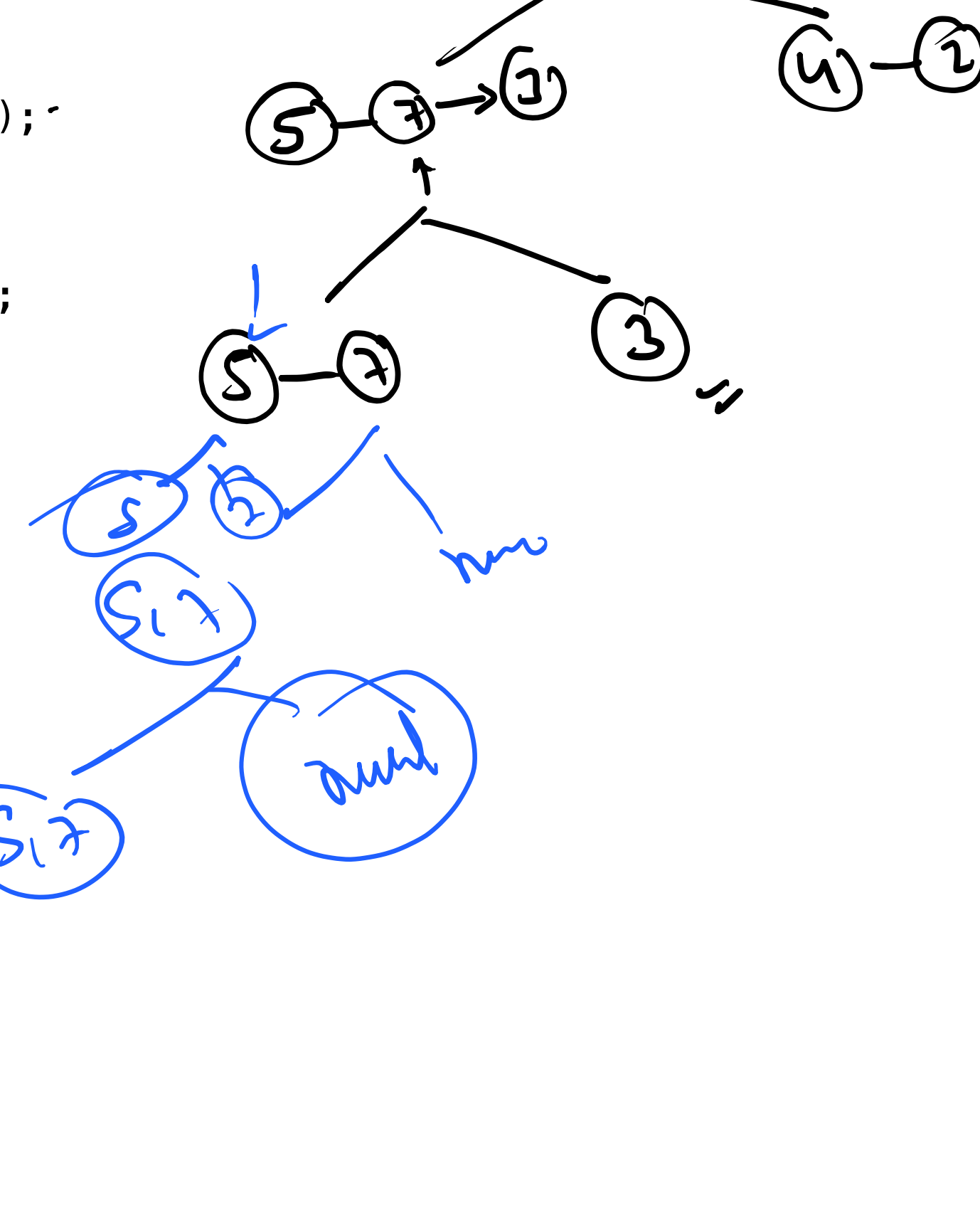
`ListNode dummy = new ListNode();`
`ListNode temp = dummy;`

While $cl_1 \neq null$ & $cl_2 \neq null$
if $cl_1.val < cl_2.val$ {
 $dummy.next = cl_1$;
 $cl_1 = cl_1.next$;
} else {
 $dummy.next = cl_2$;
 $cl_2 = cl_2.next$;
} $dummy = dummy.next$;

middle = $\frac{2+6}{2} = 4$

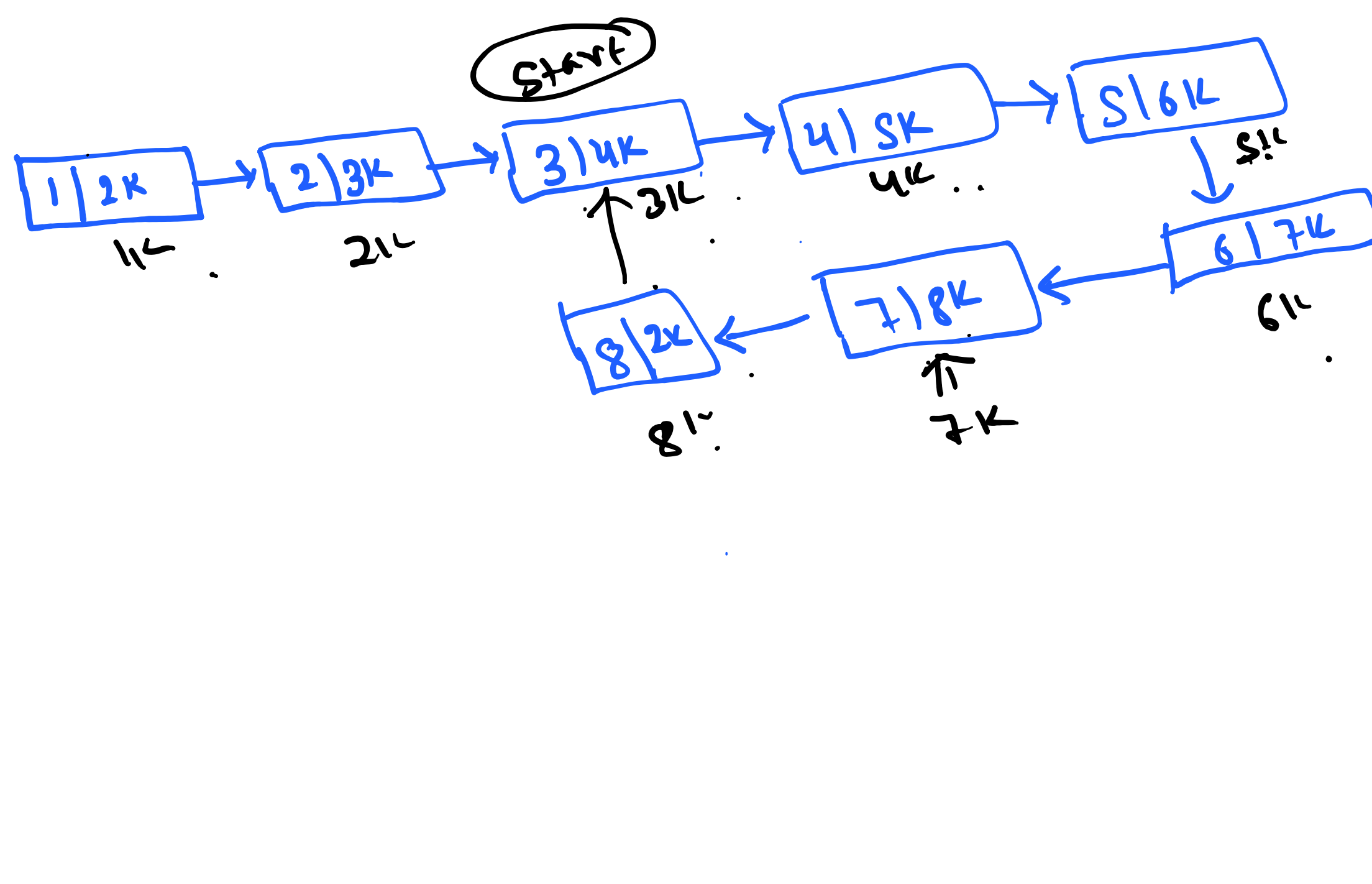
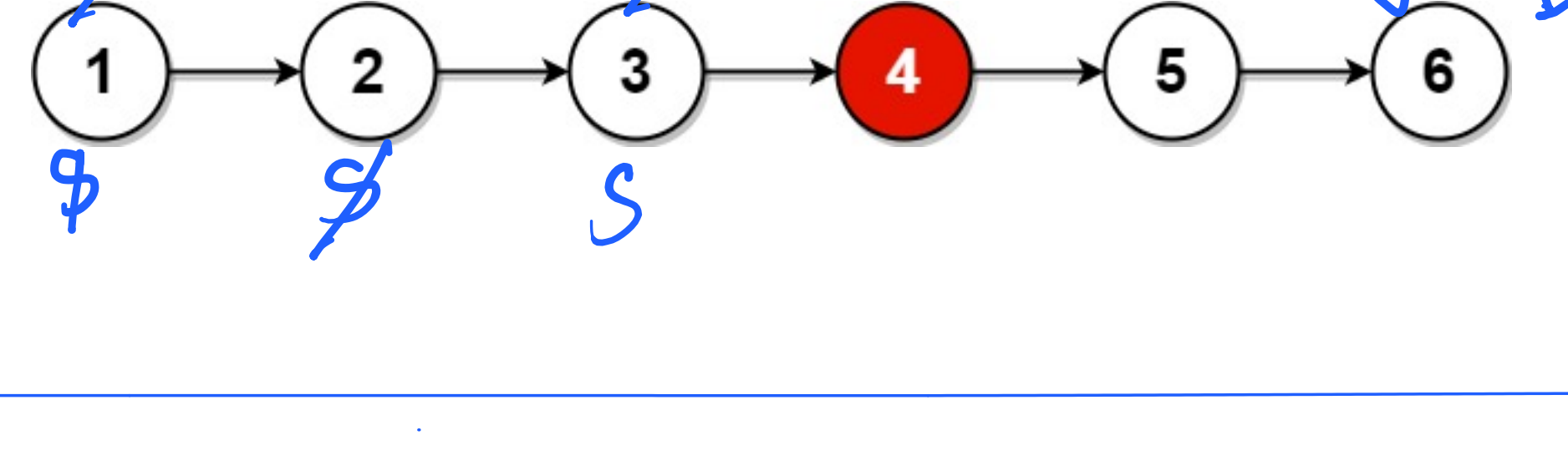


```
public ListNode sortList(ListNode head) {  
    if(head==null || head.next==null) {  
        return head;  
    }  
    ListNode mid = middleNode(head);  
    ListNode midnext = mid.next;  
    mid.next = null;  
    ListNode a = sortList(head);  
    ListNode b = sortList(midnext);  
    return mergeTwoLists(a, b);  
}
```

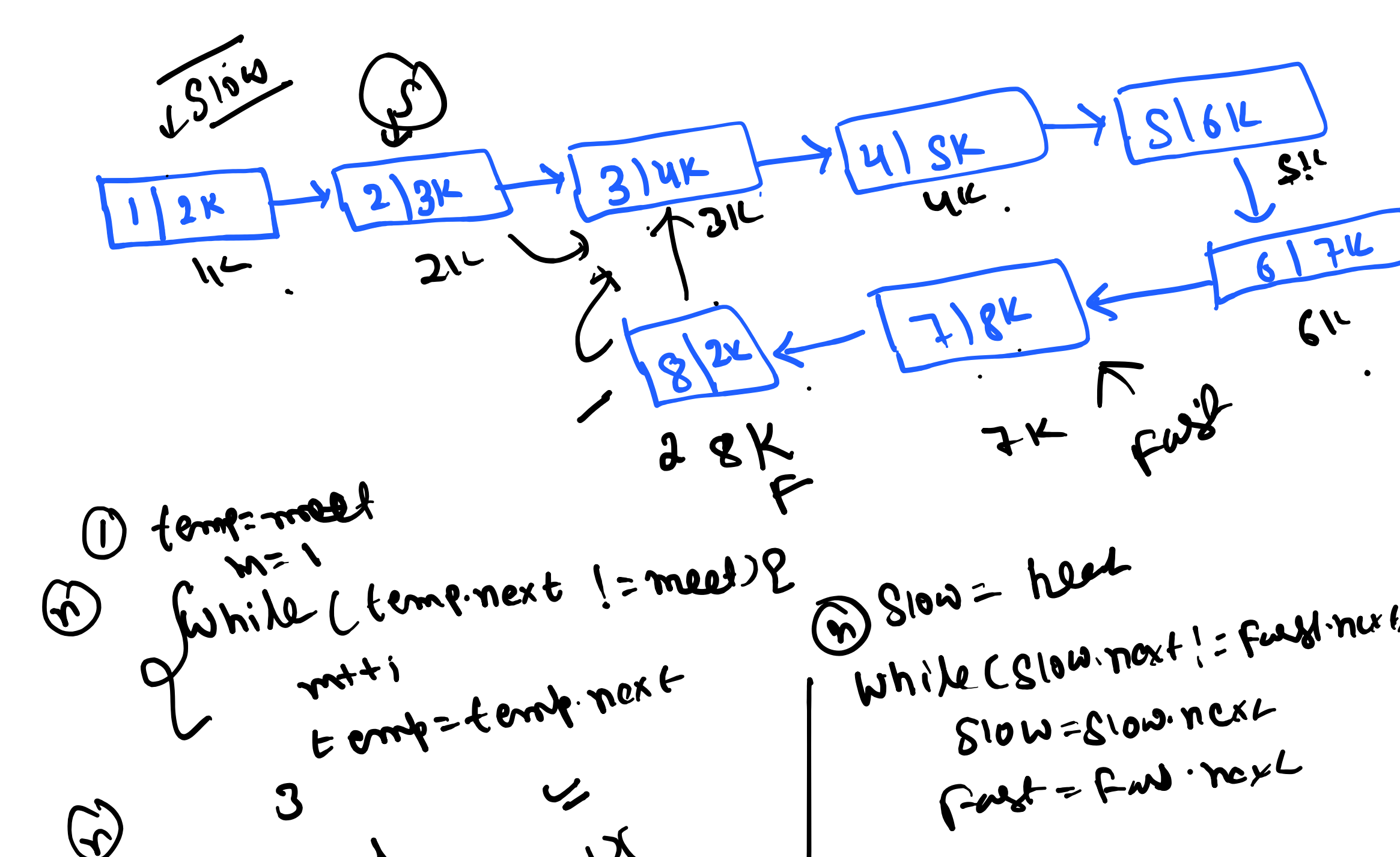


```
public ListNode middleNode(ListNode head) {  
    ListNode fast = head;  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
    }  
    return slow;  
}
```

$fast.next.next \neq null$

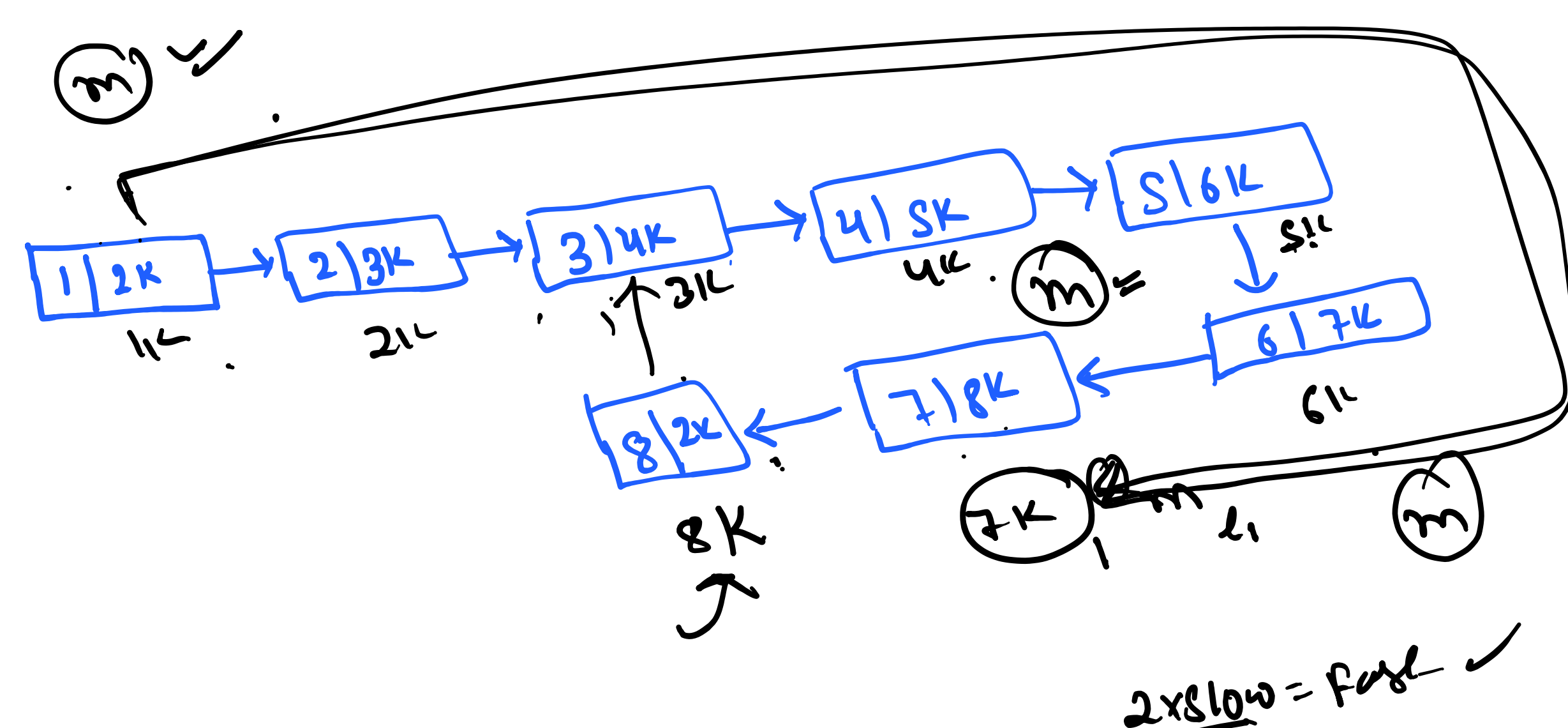
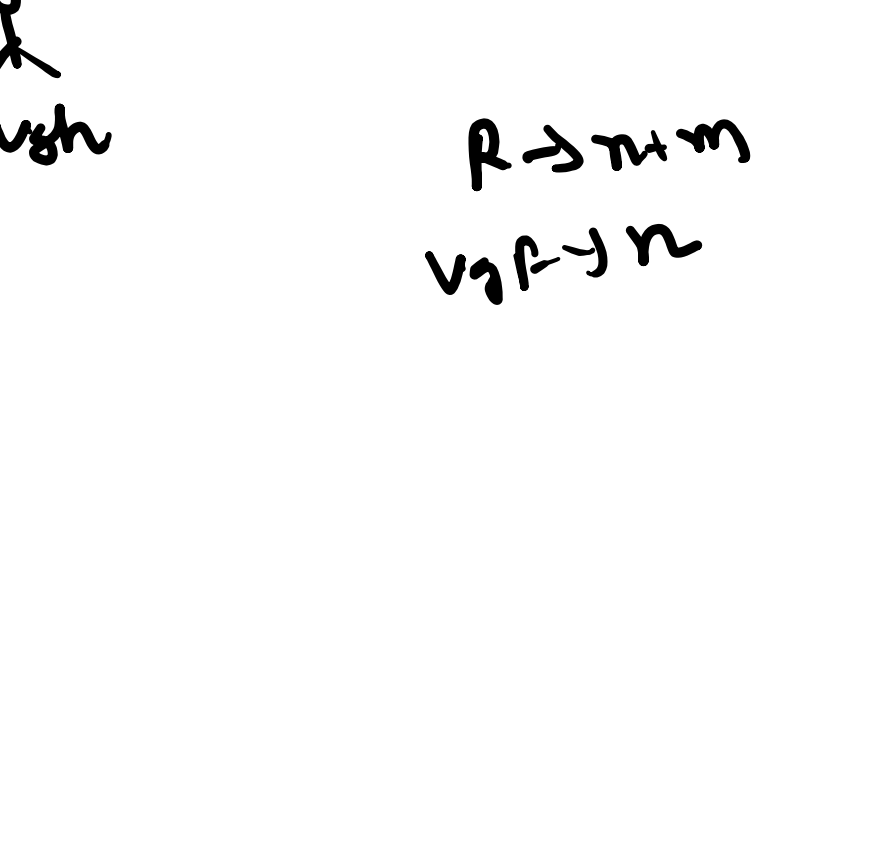


tail.next = null
while (start != null) {
 node temp = start;
 while (temp.next != null) {
 if (temp.next == start) {
 temp.next = null;
 break;
 }
 temp = temp.next;
 }
 start = start.next;
}



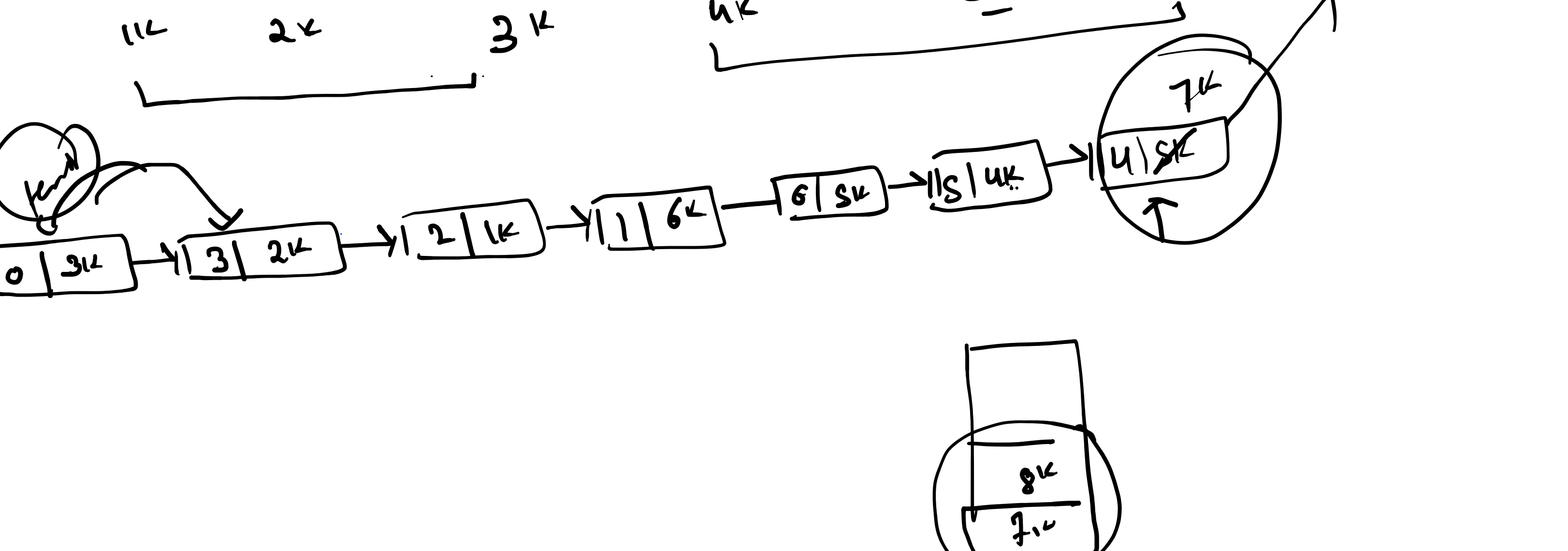
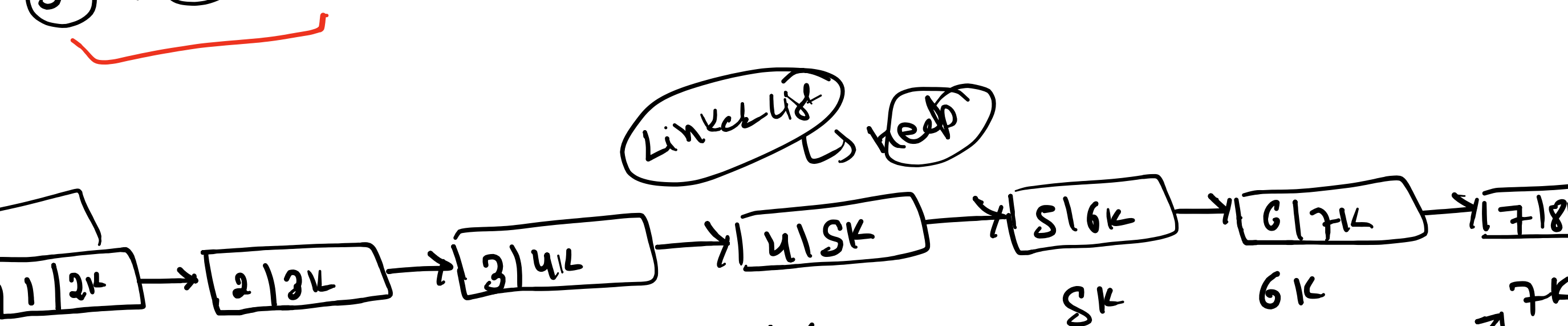
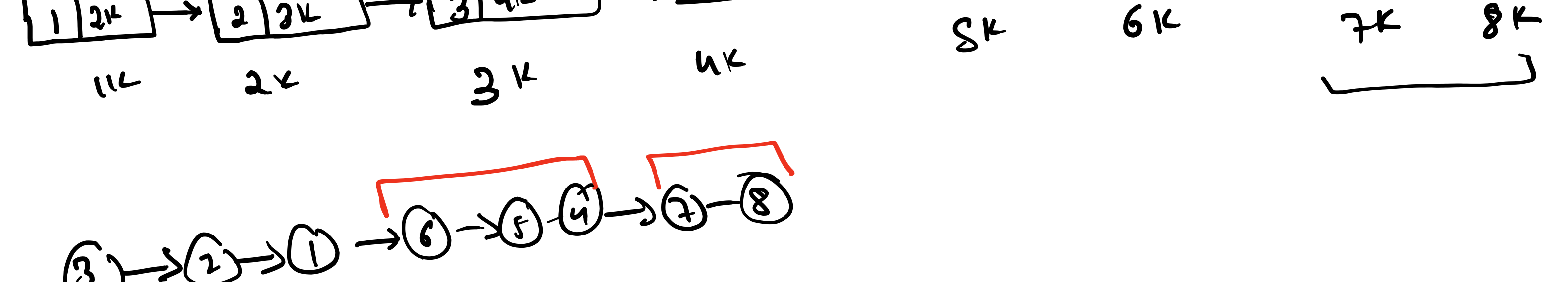
1) temp = start
2) while (temp.next != null) {
 temp = temp.next;
 if (temp.next == start) {
 temp.next = null;
 break;
 }
}

3) slow = head
while (slow.next != fast.next) {
 slow = slow.next;
 fast = fast.next;
}

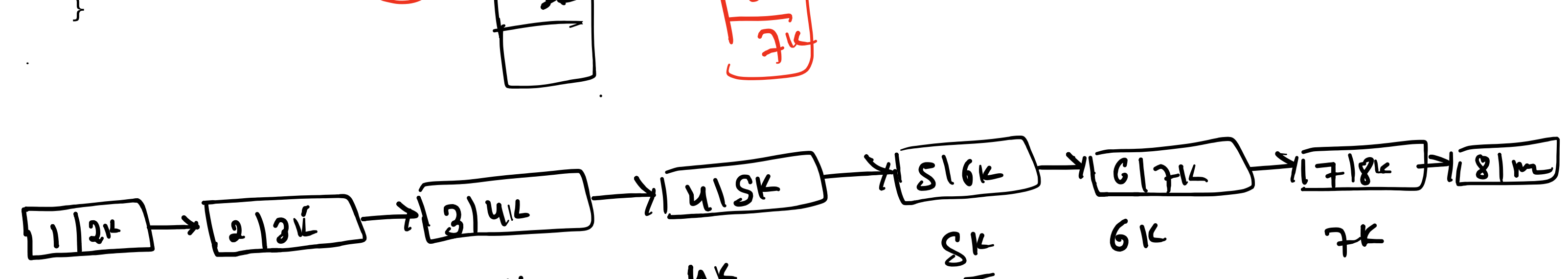


$slow = \frac{2+6}{2} = 4$
 $fast = 4$

$2 \times slow = fast$
 $2 \times d_1 = d_1 + t$
 $t = d_1$



```
public ListNode reverseKGroup(ListNode head, int k) {  
    Stack<ListNode> st = new Stack<>();  
    ListNode dummy = new ListNode();  
    ListNode temp = dummy;  
    while (head != null) {  
        st.push(head);  
        head = head.next;  
        if (st.size() == k) {  
            while (!st.isEmpty()) {  
                dummy.next = st.pop();  
                dummy = dummy.next;  
            }  
            temp = dummy;  
        }  
    }  
    return temp.next;  
}
```



```
public ListNode reverseKGroup(ListNode head, int k) {  
    Stack<ListNode> st = new Stack<>();  
    ListNode dummy = new ListNode();  
    ListNode temp = dummy;  
    while (head != null) {  
        st.push(head);  
        head = head.next;  
        if (st.size() == k) {  
            while (!st.isEmpty()) {  
                dummy.next = st.pop();  
                dummy = dummy.next;  
            }  
            temp = dummy;  
        }  
    }  
    return temp.next;  
}
```

